Programming Fundamentals

Question 1: What is a variable in C/C++?

A variable is a named storage location in the computer's memory (RAM) that holds data which can be modified during program execution. It acts as a container for data, mapped directly to a specific memory address with a specific size determined by its data type.

Question 2: Why do we use variables in programs?

We use variables to store, track, and manipulate data dynamically. Without variables, we wouldn't have a way to save user inputs, hold computational results, or re-use changing values throughout our source code. They allow programs to handle variable inputs instead of working with hardcoded values.

Question 3: What is the difference between a variable name and its value?

- **Variable Name (Identifier):** The label or alias used in your source code to access a specific memory location (e.g., `age`).
- **Variable Value:** The actual data or content physically stored inside that specific memory location (e.g., `18`).

Question 4: Which of the following is a valid variable name?

The correct answer is **b) total_marks**.

Explanation: `2num` is invalid because it starts with a number; `for` is invalid because it is a reserved language keyword; `my marks` is invalid because it contains an illegal whitespace character.

Question 5: Why is Age different from age?

C and C++ are strictly **case-sensitive** programming languages. This means identifiers containing identical letters but different capitalization (such as uppercase 'A' versus lowercase 'a') point to entirely independent allocations in memory.

Question 6: Can a variable name contain spaces? Why?

No, a variable name cannot contain spaces. The compiler treats spaces as tokens or delimiters. If you try to declare `int my age = 20;`, the compiler interprets `my` and `age` as separate tokens, resulting in a syntax error. To separate words, use snake_case (`my_age`) or camelCase (`myAge`).

Question 7: Can a variable name start with an underscore (_) ?

Yes, starting a variable name with an underscore is syntactically completely valid. However, as a rule of thumb, it's best avoided for general use because standard libraries often use leading underscores for internal implementation variables, which can create subtle naming collisions.

Question 8: What happens if two variables have the same name in the same scope?

It causes a **compilation error** (specifically a redefinition error). The compiler needs to map each name uniquely within a given scope to understand exactly where to look up data; duplicating a name creates ambiguity that the tool cannot resolve.

Question 9: Write three meaningful variable names for storing student information.

- `studentRollNumber` (or `student_roll_no`)
- `studentGpa` (or `student_gpa`)
- `studentEmail` (or `student_email`)

Question 10: Which naming convention is better and why: studentMarks or sm?

`studentMarks` is far superior. Descriptive variable names make source code self-documenting and easy to read. Cryptic names like `sm` make code bases hard to maintain, debug, and scale, especially when working in collaborative software engineering teams.

Question 11: What is a data type?

A data type is an attribute that tells the compiler what kind of data a variable holds, how much memory space to allocate for it (e.g., 4 bytes for an integer), and what kind of mathematical or logical operations can safely be performed on that data.

Question 12: Which data type is used to store whole numbers?

The `int` (integer) data type is used to store positive or negative whole numbers without fractional or decimal components.

Question 13: Which data type is used to store decimal values?

Both `float` and `double` are used to store decimal values (floating-point numbers).

Question 14: What is the difference between float and double?

- **float:** Single-precision floating-point. Typically occupies 4 bytes of memory and provides roughly 7 digits of decimal precision.
- **double:** Double-precision floating-point. Typically occupies 8 bytes of memory and offers roughly 15 digits of decimal precision, making it more accurate for complex arithmetic.

Question 15: Which data type stores a single character?

The `char` data type is used to store a single character. Character values must be enclosed within single quotes (e.g., `'A'`).

Question 16: Which data type stores True/False values?

The `bool` (boolean) data type is used, which can hold only one of two logical values: `true` or `false`.

Question 17: What data type is suitable for storing a person's name?

The `std::string` class data type (available from the `<string>` header in C++) is suitable, as it stores a sequence of characters.

Question 18: Predict the output: int age=18; cout<<age;

Output: `18`

Explanation: The variable `age` evaluates directly to its initialized value of 18, which is then sent to the standard console output stream.

Question 19: Identify the data type: 25, 3.14, 'A', true.

- `25` → `int`
- `3.14` → `double` (or `float` if explicitly cast with an 'f' suffix)
- `'A'` → `char`
- `true` → `bool`

Question 20: What happens if you store a decimal number in an int variable?

The number undergoes **truncation**. The fractional/decimal part is completely discarded, and only the whole number part is retained. For example, assigning `3.99` to an `int` yields exactly `3` without any rounding.

Question 21: What is the purpose of cin?

`cin` (character input) is the standard input stream object in C++. It is used to capture data entered by the user from an input device, usually the keyboard.

Question 22: What is the purpose of cout?

`cout` (character output) is the standard output stream object in C++. It is used to transmit and display data onto the system output terminal or screen.

Question 23: Which symbol is used with cin?

The extraction (or stream get-from) operator, written as `>>`.

Question 24: Which symbol is used with cout?

The insertion (or stream put-to) operator, written as `<<`.

Question 25: Explain: cin >> age;

This statement pauses program execution, waits for the user to type a value in the console and hit Enter, extracts that input from the keyboard stream buffer, and stores it directly inside the variable age.

Question 26: Explain: cout << age;

This statement reads the current value stored in the variable `age`, sends that value over to the standard output stream, and prints it out on the user's console screen.

Question 27: What is the purpose of endl?

endl is a stream manipulator that does two things: it inserts a newline character (' ') into the output stream to move the cursor to the next line, and it flushes the output stream buffer immediately to guarantee that all pending outputs are printed safely on the screen.

Question 28: Write a statement to input a student's marks.

```
cin >> studentMarks;
```

Question 29: Write a statement to display 'Welcome to C++'.

```
cout << "Welcome to C++";
```

Question 30: Predict the output: cout << 'Programming';

Output: Multi-character literal warning / compilation warning, outputting a numerical value like 1886151027 instead of the text.

Explanation: Single quotes (') are reserved exclusively for single character literals in C++. Long multi-character text must always be enclosed inside double quotes ("Programming"). Writing strings in single quotes triggers unexpected multi-character literal integer conversion values.

Question 31: What is an operator?

An operator is a special symbolic token that instructs the compiler to execute specific mathematical, logical, or relational transformations on one or more variables or values (operands).

Question 32: Which operator is used for addition?

The plus sign symbol (`+`).

Question 33: What is the result of 10 % 3?

Result: 1

Explanation: The modulo operator (%) evaluates the remainder of integer division. When 10 is divided by 3, the quotient is 3 and the remaining remainder left over is 1 (*10 = 3 × 3 + 1*).

Question 34: What is the result of 8 % 2?

Result: 0

Explanation: Because 8 is completely and evenly divisible by 2 (*8 = 4 × 2 + 0*), there is absolutely no remainder left over.

Question 35: Which operator checks equality?

The double equal-sign operator (`==`).

Question 36: What is the difference between = and == ?

- `=` is the **assignment operator**. It copies the value from its right-hand side and saves it into the variable container on its left-hand side (e.g., `x = 5;`).
- `==` is the **equality relational operator**. It checks whether the expression on the left equals the expression on the right and returns `true` or `false` (e.g., `if (x == 5)`).

Question 37: What is the result of 5 > 3?

Result:1 (or true)

Explanation: The relational inequality evaluates as a completely valid truth assertion, returning boolean `true` , which outputs as integer `1` in numeric print context streams.

Question 38: What does && represent?

It represents the logical **AND** operator. It returns `true` if and only if both the left operand conditions and right operand conditions evaluate to true simultaneously.

Question 39: What does || represent?

It represents the logical **OR** operator. It returns `true` if at least one of its surrounding operands or conditional expressions evaluates to true.

Question 40: What does ! represent?

It represents the logical **NOT** (unary inversion) operator. It reverses the logical evaluation of a state—turning `true` expressions into `false`, and vice versa.

Question 41: Write a program to print 'Hello World'.

```
#include <iostream>
using namespace std;

int main() {
	cout << "Hello World" << endl;
	return 0;
}
```

Question 42: Write a program to input two numbers and display their sum.

```
#include <iostream>
using namespace std;

int main() {
	int num1, num2, sum;
	cout << "Enter two numbers: ";
	cin >> num1 >> num2;

	sum = num1 + num2;
	cout << "Sum = " << sum << endl;

	return 0;
}
```

Question 43: Write a program to calculate the area of a rectangle.

```
#include <iostream>
using namespace std;

int main() {
	double length, width, area;
	cout << "Enter length and width of the rectangle: ";
	cin >> length >> width;

	area = length * width;
	cout << "Area of rectangle = " << area << endl;

	return 0;
}
```

Question 44: What formula is used to calculate Simple Interest?

The standard algebraic formula is:

$$Simple\ Interest = (P \times R \times T) / 100$$

Where **P** is the Principal amount, **R** is the annual Interest Rate percentage, and **T** is the Time period duration in years.

Question 45: Write a program to calculate Simple Interest.

```
#include <iostream>
using namespace std;

int main() {
    double principal, rate, time, si;
    cout << "Enter Principal, Rate, and Time: ";
    cin >> principal >> rate >> time;

    si = (principal * rate * time) / 100.0;
    cout << "Simple Interest = " << si << endl;

    return 0;
}
```

Question 46: Why is a temporary variable used while swapping two numbers?

If you directly copy values between two variables (e.g., a = b;), the original value inside variable a is permanently overwritten and lost before it can be copied into b. A temporary variable acts as a safe, intermediate holding box to preserve a's original value during the transfer process.

Question 47: Swap the values: a=10, b=20.

Here is the standard algorithmic sequence to perform this swap manually or inside code:

```
int temp  = a;   // temp gets 10
a = b;           // a gets 20
b = temp;        // b gets 10
```

Question 48: What formula is used to convert Celsius into Fahrenheit?

The standard mathematical translation equation is:

$$F = (C \times 9/5) + 32$$

Or, represented using clean decimal coefficients: $F = (C \times 1.8) + 32$.

Question 49: What is the Fahrenheit value of 0°C?

Value: 32°F

Explanation: Substituting 0 into the conversion formula updates it to: $F = (0 \times 1.8) + 32 = 32$.

Question 50: Why can integer division give incorrect results in the Celsius-to-Fahrenheit program?

If you write the fraction expression using integers as 9 / 5 inside your code, the compiler treats it as integer division and truncates the true value of 1.8 down into an exact integer scalar 1. Multiplying temperature scales by 1 instead of 1.8 destroys the math logic completely. To fix this behavior, you must enforce floating- point division context by adding clear decimal extensions: 9.0 / 5.0 or 9 / 5.0.